Adesso sul mio raspberry voglio definire un service che faccia questo, chiami bot_cam.py, con ambiente del virtualEnvCam

(virtualEnvCam) martons00@MartonsPi:~/Desktop/CamIpTelegram_casa $ pwd
/home/martons00/Desktop/CamIpTelegram_casa
(virtualEnvCam) martons00@MartonsPi:~/Desktop/CamIpTelegram_casa $ ls
bot_cam.py cam.py img ip_cam_01.txt ip_cam_02.txt ip_cam.py requirements.txt token.txt virtualEnvCam

Sì: per il Raspberry la soluzione giusta è creare un servizio `systemd` che usi direttamente il Python del tuo virtualenv in `ExecStart`, impostando anche `WorkingDirectory` sulla cartella del progetto. Con `systemd` conviene usare percorsi assoluti e far partire lo script col binario Python del virtual environment, senza fare `source activate` dentro il service.[1] [2] [3]

File service

Nel tuo caso, visto che il progetto è in `/home/martons00/Desktop/CamIpTelegram_casa` e il venv è `virtualEnvCam`, puoi creare un service tipo questo. `Restart=on-failure` è utile perché fa ripartire il bot se crasha.[4] [5]

Crea il file:

```
sudo nano /etc/systemd/system/bot_cam.service
```

e incolla:

```
[Unit]
Description=Telegram Camera Bot
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=martons00
Group=martons00
WorkingDirectory=/home/martons00/Desktop/CamIpTelegram_casa
ExecStart=/home/martons00/Desktop/CamIpTelegram_casa/virtualEnvCam/bin/python /home/
Restart=on-failure
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

Attivazione

Dopo aver salvato il file, devi ricaricare `systemd`, avviare il servizio e abilitarlo all'avvio del Raspberry. `enable` lo rende persistente al boot, mentre `start` lo lancia subito.[6] [7]

```
sudo systemctl daemon-reload
sudo systemctl enable bot_cam.service
sudo systemctl start bot_cam.service
```

Comandi utili

Per verificare che funzioni, controlla lo stato del servizio e poi i log live con `journalctl`. `systemd` mostra chiaramente il comando eseguito, il PID e gli errori Python emessi dal bot.[5] [6]

```
sudo systemctl status bot_cam.service
journalctl -u bot_cam.service -f
```

Nota pratica

Poiché il tuo script legge file locali come `token.txt`, `ip_cam_01.txt`, `ip_cam_02.txt` e salva immagini in `./img`, `WorkingDirectory=/home/martons00/Desktop/CamIpTelegram_casa` è importante, altrimenti i path relativi potrebbero non funzionare. L'uso di path assoluti in `ExecStart` e di una working directory esplicita è una pratica raccomandata per servizi Python in virtualenv.[2] [3]

Se vuoi aggiornarlo

Ogni volta che modifichi il file `.service`, devi fare un nuovo `daemon-reload` e poi riavviare il servizio. Se modifichi solo `bot_cam.py`, in genere basta il restart del servizio.[6]

```
sudo systemctl daemon-reload
sudo systemctl restart bot_cam.service
```

Se vuoi, ti preparo anche una versione un po' più robusta con:

- `Environment=PYTHONUNBUFFERED=1`
- log più puliti,
- avvio ritardato di qualche secondo dopo la rete.

⁂

1. https://stackoverflow.com/questions/37211115/how-to-enable-a-virtualenv-in-a-systemd-service-unit
2. https://blog.merzlabs.com/posts/python-autostart-systemd/

3. https://learn.adafruit.com/python-virtual-environment-usage-on-raspberry-pi/automatically-running-at-boot
4. https://gist.github.com/ricferr/90583f608f0b0ae9c3cf6833be04ab85
5. https://github.com/torfsen/python-systemd-tutorial
6. https://piotrnowicki.com/posts/2024-07-25/create-linux-auto-restartable-service-on-file-change/
7. https://johnsturgeon.me/2024/07/01/python-script-as-service/
8. https://tderflinger.com/using-systemd-to-start-a-python-application-with-virtualenv
9. https://stackoverflow.com/questions/60959081/python-script-in-systemd-virtual-environment-or-real-environment
10. https://rolisz.ro/2023/setting-up-systemd/
11. https://forums.raspberrypi.com/viewtopic.php?t=330651
12. https://www.baeldung.com/linux/systemd-service-periodic-restart
13. https://www.reddit.com/r/learnpython/comments/1n3hotq/how_to_run_a_program_when_turn_on_the_raspberry/
14. http://iltabiai.github.io/raspberry pi/python/linux/ubuntu/telegram/2020/05/08/python-systemd.html
15. https://gist.github.com/dunkelstern/5bfe7414fc0b7e8a9f6e1c4c78fd2543